



INSTITUTO POLITÉCNICO NACIONAL ESCUELA SUPERIOR DE CÓMPUTO



INGENIERÍA EN SISTEMAS COMPUTACIONALES

DOCUMENTO TÉCNICO

Alumno

Nava Villar Eric

2022630293

Profesor:

Gabriel Hurtado Aviles

1. Objetivo del documento

Este documento describe las decisiones de diseño y la implementación técnica del sistema YT Stream BT, con énfasis particular en el protocolo de comunicación sobre Bluetooth y en la arquitectura Cliente-Servidor. Está dirigido a quien deba dar mantenimiento, extender o evaluar técnicamente el proyecto.

2. Arquitectura general

2.1 Modelo Cliente-Servidor de rol seleccionable

El sistema se implementa como una única aplicación Android (un solo módulo Gradle, un solo APK, com.example.extra_navavillareric). En tiempo de ejecución, cada instalación elige su rol — Servidor o Cliente — desde la pantalla de inicio (HomeScreen). Ambos roles comparten la misma Activity (MainActivity), la misma instancia de BluetoothManager y el mismo modelo de mensajes (BTProtocol).

Alternativas consideradas

Opción	Ventaja	Por qué no se eligió / por qué se eligió
Dos apps independientes (Servidor.apk y Cliente.apk)	Separación total de responsabilidades; cada APK solo incluye el código que necesita.	Duplicaría el protocolo de mensajes y el modelo de datos en dos codebases, con alto riesgo de que ambos lados se desincronicen al modificar el protocolo. No se eligió.
Un solo APK con selección de rol en tiempo de ejecución (elegido)	Un único protocolo compartido por definición; imposible que Cliente y Servidor queden en versiones incompatibles del framing de mensajes.	Se eligió: reduce superficie de error de protocolo a costa de incluir en el mismo binario código de ambos roles (costo de tamaño de APK aceptable para el alcance del proyecto).

2.2 Capas del sistema

Capa	Responsabilidad	Archivos principales
UI (Compose)	Pantallas de Servidor/Cliente, reproductor, temas Guinda/Azul claro-oscuro	MainActivity.kt, ui/theme/*
Transporte Bluetooth	Framing de mensajes, envío/recepción ordenada por RFCOMM, estado de conexión	BluetoothManager.kt
Protocolo de transferencia de archivo	Fragmentación, checksums y reensamblado del video	TransferProtocol.kt (TransferBlock)
Extracción de YouTube	Búsqueda y resolución de streams reales contra YouTube	YouTubeSearchManager.kt, OkHttpDownloader.kt, NewPipeBootstrap.kt
Modelo de datos	Representación serializable de resultados de búsqueda	VideoModel.kt

3. Protocolo de comunicación Bluetooth

3.1 Transporte

- Perfil: Bluetooth Classic (BR/EDR), perfil SPP (Serial Port Profile) sobre sockets RFCOMM.
- UUID de servicio: 00001101-0000-1000-8000-00805F9B34FB (UUID estándar reservado para SPP).
- El Servidor abre un BluetoothServerSocket vía listenUsingRfcommWithServiceRecord y acepta una única conexión entrante (accept()) bloqueante en una corrutina de IO).
- El Cliente crea el socket con createRfcommSocketToServiceRecord(UUID) contra un BluetoothDevice ya vinculado y llama a connect().

Se eligió Bluetooth Classic/RFCOMM en lugar de BLE (Bluetooth Low Energy) porque RFCOMM ofrece un socket de flujo (stream) full-duplex y confiable en orden, sin el límite de tamaño de paquete (MTU) tan reducido de BLE — indispensable para transferir un archivo de video de varios megabytes de forma práctica.

3.2 Formato de mensaje (framing)

Todo mensaje enviado por el socket sigue un encabezado fijo de 5 bytes seguido del payload:

Campo	Tamaño	Tipo	Descripción
type	1 byte	entero sin signo	Identifica el tipo de mensaje (ver tabla 3.3).
length	4 bytes	entero (big-endian, ByteBuffer por defecto)	Longitud en bytes del payload que sigue.
payload	length bytes	binario	Contenido específico según el tipo de mensaje.

La lectura del lado receptor (listenToSocket) usa una función readExactly que bloquea hasta completar exactamente el número de bytes esperado, evitando lecturas parciales típicas de sockets de flujo TCP-like.

3.3 Tipos de mensaje (BTProtocol)

Constante	Valor	Dirección típica	Propósito
TYPE_SEARCH_QUERY	0x01	Cliente → Servidor	Texto de búsqueda o URL de YouTube pegada por el usuario.
TYPE_SEARCH_RESULTS	0x02	Servidor → Cliente	Lista de VideoModel serializada en JSON con los resultados.
TYPE_VIDEO_CHUNK	0x03	—	Definido en el protocolo; no utilizado por la implementación actual (reemplazado por TYPE_TRANSFER_BLOCK).
TYPE_CONTROL_CMD	0x04	Ambas direcciones	Comandos de texto: solicitud de stream (STREAM:<id>) y señalización de sesión (ver 3.5).
TYPE_TRANSFER_BLOCK	0x05	Servidor → Cliente	Un bloque de datos binarios del archivo de video (ver 3.4).

Constante	Valor	Dirección típica	Propósito
TYPE_TRANSFER_ACK	0x06	Cliente → Servidor	Confirmación de recepción de un bloque (índice recibido).
TYPE_TRANSFER_NACK	0x07	Cliente → Servidor	Bloque recibido con checksum inválido o malformado.

3.4 Formato de TransferBlock

El payload de un mensaje TYPE_TRANSFER_BLOCK es a su vez un sub-formato binario propio (TransferProtocol.kt), con su propio encabezado y checksum, independiente del framing genérico de la sección 3.2:

Campo	Tamaño	Descripción
magic	4 bytes	Literal "BTB1" (ASCII); permite detectar payloads corruptos o de otro protocolo.
sessionLength	2 bytes (short)	Longitud en bytes del identificador de sesión que sigue.
sessionId	sessionLength bytes	Identificador de sesión de streaming, formato "sess- <timestampMs>".
blockIndex	4 bytes (int)	Índice secuencial del bloque dentro de la sesión, empezando en 0.
offset	8 bytes (long)	Desplazamiento acumulado de bytes ya enviados antes de este bloque.
isLast	1 byte	1 si es el último bloque de la sesión (bloque vacío de cierre), 0 en caso contrario.
dataSize	4 bytes (int)	Tamaño en bytes del fragmento de video que sigue.
checksum	8 bytes (long)	CRC32 calculado sobre los bytes de datos, para detectar corrupción.
data	dataSize bytes	Fragmento binario del archivo de video (hasta 16 KB por bloque).

En la recepción (TransferBlock.fromByteArray), el checksum se recalcula sobre los datos recibidos y se compara contra el que viaja en el encabezado; si no coincide, el método devuelve null y el Cliente responde con TYPE_TRANSFER_NACK en lugar de escribir el bloque al archivo.

3.5 Secuencia de control de una sesión de streaming

Comandos de texto enviados como TYPE_CONTROL_CMD que orquestan una sesión de transferencia:

```

Cliente → Servidor:  STREAM:<videoId>
Servidor → Cliente:  SESSION_START:<sessionId>
Servidor → Cliente:  SIZE:<totalBytesEsperados>
Servidor → Cliente:  START_STREAM
Servidor → Cliente:  N x TYPE_TRANSFER_BLOCK (isLast=false)
Servidor → Cliente:  1 x TYPE_TRANSFER_BLOCK (isLast=true, sin datos)

```

Servidor → Cliente: `END_STREAM`

Camino de error (en cualquier punto en que el Servidor no pueda obtener o servir el video):

Servidor → Cliente: `ERROR_STREAM`

Al recibir `SESSION_START`, el Cliente borra cualquier archivo temporal previo (`bt_video.mp4` en el caché de la app) y reinicia sus contadores de progreso, de forma que cada nueva sesión empieza desde un estado limpio.

3.6 Entrega ordenada garantizada

Problema detectado y corregido: la implementación original de `sendData()` lanzaba una corrutina nueva (`scope.launch`) por cada llamada, protegida únicamente por un bloque `synchronized`. Eso impedía que dos envíos se entrelazaran a nivel de bytes, pero no garantizaba que el orden en que las corrutinas obtenían el lock coincidiera con el orden en que fueron invocadas — bajo carga (el bucle de envío de bloques invoca `sendData` docenas de veces por segundo), el planificador de `Dispatchers.IO` podía servir los bloques del video fuera de secuencia, corrompiendo el archivo `.mp4` reensamblado en el Cliente.

Solución implementada:

- Se sustituyó el lock `synchronized` por un `kotlinx.coroutines.sync.Mutex`.
- Se añadió una función suspend `sendDataOrdered(type, data)` que realiza la escritura directamente dentro de la corrutina que la invoca (sin lanzar una corrutina adicional), protegida por el `Mutex`.
- Toda la secuencia de streaming (`SESSION_START`, `SIZE`, `START_STREAM`, cada bloque, y `END_STREAM`) se envía ahora con `sendDataOrdered` desde la misma corrutina secuencial, lo que garantiza que el orden de escritura en el socket coincide exactamente con el orden en que los datos se generan.
- `sendData(type, data)` se conserva como variante "fire-and-forget" (lanza su propia corrutina internamente) para mensajes sueltos disparados desde la UI que no forman parte de una secuencia con restricción de orden (por ejemplo, el envío inicial de una búsqueda).

3.7 Manejo de desconexión

`listenToSocket()` corre en un bucle de lectura bloqueante; cualquier `IOException` (socket cerrado, dispositivo fuera de rango) se captura y actualiza `connectionState` a "Desconectado", reflejado en la UI de ambos roles mediante un `StateFlow` observado con `collectAsState()`. La reconexión automática no está implementada: el usuario debe volver a iniciar el rol correspondiente manualmente tras una pérdida de conexión.

4. Búsqueda y extracción de video

4.1 De proxies públicos a extracción directa

La primera versión del sistema dependía de instancias públicas de terceros de los proyectos Piped e Invidious (proxies HTTP que reexponen la API interna de YouTube). En la práctica, estas instancias resultaron poco confiables: caídas, resultados desactualizados y fallas sistemáticas en la extracción del stream reproducible. Se reemplazaron por `NewPipeExtractor`, la biblioteca de extracción que usa la propia aplicación `NewPipe`, que consulta a YouTube de forma directa en vez de depender de un tercero intermedio.

Componente	Rol
OkHttpDownloader	Implementación del Downloader que <code>NewPipeExtractor</code> requiere para realizar sus propias peticiones HTTP, respaldada por <code>OkHttp</code> .

Componente	Rol
NewPipeBootstrap	Inicialización única y segura entre hilos de NewPipe.init(), requerida antes de cualquier uso de la librería.
YouTubeSearchManager.search(query)	Detecta si la entrada es una URL de YouTube (usa StreamInfo.getInfo directo) o texto libre (usa el SearchExtractor filtrado a contenido de tipo video).
YouTubeSearchManager.extractStreamUrl(videoId)	Dado un id de video, resuelve la URL descargable directa del stream progresivo elegido.

4.2 Algoritmo de selección de calidad de stream

1. Se obtiene StreamInfo del video vía NewPipeExtractor.
2. Se filtran los streams con deliveryMethod == PROGRESSIVE_HTTP (excluye DASH/HLS, que este proyecto no puede reproducir con su enfoque de descarga progresiva simple).
3. Se prioriza la lista videoStreams (streams "muxed", con audio incluido) sobre videoOnlyStreams (sin audio); solo se recurre a estos últimos si no hay ningún muxed disponible.
4. Dentro de la lista priorizada, se elige la resolución numéricamente más baja disponible, para minimizar el tiempo de transferencia por Bluetooth.

Esta preferencia por streams muxed de baja resolución no es solo una optimización de ancho de banda: los formatos muxed heredados de YouTube (como el equivalente al itag 18, 360p) tienden a seguir funcionando de forma más estable frente a las restricciones de poToken/SABR que los formatos DASH de alta resolución, que YouTube protege de forma más agresiva.

5. Manejo de errores extremo a extremo

5.1 Propagación de errores de extracción

Problema detectado y corregido: cuando el Servidor no lograba obtener el stream (video no disponible, fallo de extracción, etc.) enviaba ERROR_STREAM, pero el Cliente solo actualizaba una variable de estado (searchStatus) que pertenece a la pantalla de búsqueda — pantalla que ya no era visible porque el usuario había sido llevado a la pantalla del reproductor al seleccionar el video. El resultado era una pantalla negra permanente sin ninguna indicación del error real.

Solución: al recibir ERROR_STREAM, el Cliente ahora también restablece playingVideo a null, devolviendo al usuario a la pantalla de búsqueda donde el mensaje de error sí es visible.

Adicionalmente, se detectó que el sentinel de error en los resultados de búsqueda (VideoModel con id == "error") nunca era producido por ningún camino del código original, por lo que la rama de manejo de error en el Cliente era código muerto. YouTubeSearchManager ahora sí genera ese sentinel — con un mensaje descriptivo en el campo snippet — cuando la búsqueda o la extracción fallan por completo.

5.2 Errores asíncronos de ExoPlayer

Problema detectado y corregido: un archivo incompleto o corrupto puede hacer que ExoPlayer.prepare() falle de forma asíncrona en su hilo interno de reproducción. Sin un listener de errores, esa falla nunca se reflejaba en la interfaz — la app ya había marcado playerInitialized = true (porque prepare() en sí no lanza una excepción síncrona) y solo se veía un PlayerView negro sin ningún fotograma.

Solución: se agregó un `Player.Listener` con override de `onPlayerError` que actualiza el estado de reproducción visible en pantalla (`playbackStatus`) con el código de error real reportado por `ExoPlayer`.

6. Resumen de decisiones de diseño

Decisión	Justificación
Un solo APK con rol seleccionable en vez de dos apps	Un único protocolo de mensajes compartido; imposible que Cliente y Servidor diverjan de versión.
RFCOMM/SPP (Bluetooth Classic) en vez de BLE	Socket de flujo confiable y en orden, sin el límite de MTU reducido de BLE; necesario para transferir archivos de varios MB.
Bloques de 16 KB con pausa de 25 ms entre envíos	Calibrado empíricamente para estabilidad del radio Bluetooth en los dispositivos de prueba (Samsung S10 / Tab A8), evitando saturar el buffer del socket.
Umbral de búfer de 500 KB antes de iniciar reproducción	Suficiente en la práctica para que <code>Media3/ExoPlayer</code> localice el átomo moov del MP4 y comience a decodificar sin fallar por datos insuficientes.
CRC32 + bytes mágicos en cada bloque (formato BTB1)	Detecta corrupción a nivel de bloque individual sin depender únicamente de la fiabilidad del socket RFCOMM subyacente.
Mutex + envío secuencial (<code>sendDataOrdered</code>) en vez de <code>synchronized</code> + corrutinas paralelas	<code>synchronized</code> solo evitaba el entrelazado de bytes, no garantizaba el orden de llegada; el Mutex con envío secuencial sí lo garantiza.
<code>NewPipeExtractor</code> en vez de proxies <code>Piped/Invidious</code>	Elimina la dependencia de disponibilidad de terceros; consulta a YouTube de forma directa con una librería activamente mantenida.
Preferencia por streams muxed de baja resolución	Minimiza el tiempo de transferencia por Bluetooth y favorece formatos con mayor probabilidad de servirse sin requerir <code>poToken</code> .

7. Consideraciones de seguridad

- El canal Bluetooth RFCOMM no está cifrado a nivel de aplicación; se apoya únicamente en el emparejamiento y cifrado de enlace que provee el propio stack Bluetooth del sistema operativo.
- El campo `isSafe` de `VideoModel` está definido en el modelo de datos para una futura verificación de fuente segura, pero actualmente no se calcula ni se muestra en la interfaz (ver limitaciones).
- Las peticiones HTTP salientes del Servidor (extracción y descarga del video) usan siempre HTTPS.

8. Limitaciones conocidas y trabajo futuro

- Sin caché de videos en el Servidor: cada solicitud vuelve a descargar el video completo desde Internet, incluso si ya se sirvió antes.
- Sin reconexión automática tras pérdida de enlace Bluetooth.
- Sin persistencia de historial ni de favoritos en el Cliente.
- Sin modo de bajo consumo explícito (solo audio).
- Sin advertencia de fuente no verificada visible en la interfaz.
- El modo oscuro no se inicializa automáticamente desde la configuración del sistema al abrir la app (solo mediante el interruptor manual).

- Dependencia inherente de la estabilidad de la extracción de YouTube (poToken/SABR), fuera del control de este proyecto.

9. Referencias

- NewPipeExtractor — <https://github.com/TeamNewPipe/NewPipeExtractor>
- Especificación Bluetooth SPP (Serial Port Profile) — Bluetooth SIG
- Media3 / ExoPlayer — <https://developer.android.com/media/media3>
- Documentación de permisos Bluetooth en Android 12+ — developer.android.com/guide/topics/connectivity/bluetooth/permissions